AI EDA 课程实验 作业提高要求报告

学生提交模板

请使用本报告说明两项作业的设计过程、验证结果和性能数据。报告须与代码一同提交；所有数值应能由保存的运行记录、教师 TB 或 PPA 报告追溯。

提交人信息

项目	填写内容	项目	填写内容
姓名	郑珂	学号	2022012348
班级	AI EDA 2026	提交日期	2026-11-12
GitHub Fork 链接	https://github.com/2022012348/THU_AI_EDA_2026-hw	提交版本或 commit	7421958（main）
代码提交方式	GitHub 公开仓库（含 run summary、教师 TB 报告与通过的 RTL 快照；完整 runs/ 14 GB 未入库）	运行环境	macOS arm64；Python 3.11；iverilog 13.0；yosys 0.70；OpenSTA 3.1.0；模型 DeepSeek-V4.1-Flash-1017（Paratera）

代码提交要求

- 每位学生必须同时提交本报告文件和本人开发的项目代码。代码可通过在课程 GitHub 项目上 Fork 后提交链接，也可以将完整项目代码压缩后与本报告一起提交。
- GitHub 提交应给出可访问的仓库链接和用于复现的 commit；压缩包提交应保留目录结构、依赖说明和运行记录。无论采用哪种方式，均不得删除教师 Spec、TB、评分脚本或伪造测试结果。
- 请在报告中说明复现入口，例如启动命令、目标项目、模型配置、教师 TB 命令或工作台运行 ID。不要在报告、代码仓库或压缩包中包含 API Key、SSH 密码或私钥。

作业一 AES 128 加密核 Prompt 设计

任务概述：为单模块 aes128 编写可执行 Prompt，生成满足接口、AES 字节序、轮函数、密钥扩展、同步复位与握手要求的 SystemVerilog，并以教师 AES TB 验收。功能通过后，再比较面积、功耗和 token 消耗。

设计的 Prompt

实现单模块 AES-128 加密核 aes128（FIPS-197，只做加密），用于 45nm 标准单元库下的面积/功耗对比实验。

最终回答**只能包含一个 module aes128 ... endmodule 的完整 SystemVerilog 源码**：不要 Markdown 说明、不要推导过程、不要 testbench、不要第二个模块。不得出现 include、$readmemh、DPI、IP 例化或外部文件依赖。

一、接口（逐字符一致，不得改动）

```
module aes128 (
    input  logic         clk,
    input  logic         rst_n,
    input  logic         start,
    input  logic [127:0] key,
    input  logic [127:0] plaintext,
    output logic         ready,
    output logic         done,
    output logic [127:0] ciphertext
);
```

ready、done、ciphertext 必须是寄存器输出，端口不得在模块体内重复声明。

二、字节序（先固化，后续所有逻辑以此为准）

- 128-bit 向量按 FIPS-197 大端：[127:120] 是 byte 0，[7:0] 是 byte 15。
- 状态按列优先存成 4 个 32-bit 列寄存器 s0..s3，s_c 保存第 c 列，且 s_c[31:24] 是该列第 0 行：
  - 接收时（**最容易错的一步，务必逐字照抄**）：列号与位段是**同向**的——第 0 列占最高 32 位：
    s0 <= plaintext[127:96] ^ key[127:96];
    s1 <= plaintext[95:64] ^ key[95:64];
    s2 <= plaintext[63:32] ^ key[63:32];
    s3 <= plaintext[31:0] ^ key[31:0]。
    注意 plaintext[31:0] 是**最后一列**（byte 12~15），把它当成第 0 列会让整个状态转置式错位，密文全错而握手时序却完全正常（这种错误极难从波形上看出来）。
  - 输出时：ciphertext <= {col0, col1, col2, col3}，即 **col0 放在最高 32 位**（ciphertext[127:96] = col0）；写成 {col3, col2, col1, col0} 会让整条结果字节序颠倒。
- 轮密钥字：w0 = key[127:96]，w1 = key[95:64]，w2 = key[63:32]，w3 = key[31:0]；每个字的最高字节是该字第 0 字节。

三、握手、复位与时序

1. rst_n 是**同步低有效**复位，只在 clk 上升沿生效（禁止 negedge rst_n）。复位后空闲：ready=1、done=0。
2. 空闲时 start=1 的上升沿锁存 key/plaintext 并开始加密；从下一拍起 ready=0，忙碌期间忽略 start。
3. done 只高**一个周期**，该周期 ciphertext 有效、ready=0；下一拍 done=0、ready=1，且当拍能立即接受新 start（背靠背，无气泡）。
4. 从接收 start 的沿（不含）到 done=1 的沿（含）不超过 200 拍。
5. 全部寄存器同步复位；不得有 initial、#delay、锁存器、多驱动。

四、算法

- 初始轮：s = plaintext ^ {w0,w1,w2,w3}。
- 第 1–9 轮：SubBytes → ShiftRows → MixColumns → AddRoundKey；第 10 轮：SubBytes → ShiftRows → AddRoundKey。
- **每次加密对每个字节只做一次 SubBytes。**sb0..sb3 存的就是 SubBytes 之后的结果；sr0..sr3 是 sb 的 ShiftRows，**已经包含 SubBytes。**因此：
  - 第 1–9 轮的 MixColumns 输入是 sr，**不要再对 sr 调 sbox**；
  - 第 10 轮的密文是 ciphertext <= {sr0, sr1, sr2, sr3} ^ {w0, w1, w2, w3}（col0 在最高位），**同样不要再调 sbox。**

  写成 c0 = {sbox(sr0[31:24]), ...} 就是对已置换数据再置换一次：结果全错，同时多出 16 个 S-box，面积会从约 35 k 涨到约 52 k（一眼可辨）。
- ShiftRows：out[r][c] = in[r][(c+r) mod 4]。用列寄存器表示时：sr0 = {sb0[31:24], sb1[23:16], sb2[15:8], sb3[7:0]}，sr1 = {sb1[31:24], sb2[23:16], sb3[15:8], sb0[7:0]}，sr2 = {sb2[31:24], sb3[23:16], sb0[15:8], sb1[7:0]}，sr3 = {sb3[31:24], sb0[23:16], sb1[15:8], sb2[7:0]}。
- MixColumns：out[0] = 2·b0 ^ 3·b1 ^ b2 ^ b3，out[1] = b0 ^ 2·b1 ^ 3·b2 ^ b3，out[2] = b0 ^ b1 ^ 2·b2 ^ 3·b3，out[3] = 3·b0 ^ b1 ^ b2 ^ 2·b3，其中 2·b = xtime(b) = {b[6:0],1'b0} ^ (b[7] ? 8'h1b : 8'h00)，3·b = xtime(b) ^ b。

  易错点：3·b1 必须写成 xtime(b1) ^ b1，绝不能写成 b1 ^ b1（那是 0）。每个 xtime 结果只算一次并在 4 个输出字节间共享。

  直接照抄这个形状（以列 0 的 a0..a3 为例，mc0 的 4 个字节依次是 out0..out3）：

```
wire [7:0] xa0 = xtime(a0), xa1 = xtime(a1), xa2 = xtime(a2), xa3 = xtime(a3);
wire [31:0] mc0 = { xa0 ^ (xa1 ^ a1) ^ a2 ^ a3,
                    a0 ^ xa1 ^ (xa2 ^ a2) ^ a3,
                    a0 ^ a1 ^ xa2 ^ (xa3 ^ a3),
                    (xa0 ^ a0) ^ a1 ^ a2 ^ xa3 };
```

  其余三列照抄这个形状，只把 a 换成该列自己的字节名（列 1 用 b0..b3 与 xb0..xb3，依此类推）。注意 out3 以 (xtime(b0) ^ b0) 开头、以 xtime(b3) 结尾，写错这一行会让每列最后一个字节全错而握手正常。

  边界（务必遵守）：s_c 本身就是**一列**的 4 个字节。做第 c 列的 MixColumns 时，把这 4 个字节记作 b0=s_c[31:24]、b1=s_c[23:16]、b2=s_c[15:8]、b3=s_c[7:0]，四个输出只允许引用它们：m0 = 2·b0 ^ 3·b1 ^ b2 ^ b3、m1 = b0 ^ 2·b1 ^ 3·b2 ^ b3、m2 = b0 ^ b1 ^ 2·b2 ^ 3·b3、m3 = 3·b0 ^ b1 ^ b2 ^ 2·b3，其中 3·bx = xtime(bx) ^ bx。

  每一列的四个输出只能出现该列自己的字节；**不允许**出现相邻列 s_{c+1}、s_{c-1} 的字节（把 4 个列字当成同一列的 4 个字节、在 m0 里混进别的列的字节，是隐蔽错误：握手正常、密文全错、面积看不出异常）。
- 密钥扩展：w[i] = w[i-4] ^ temp；i%4==0 时 temp = SubWord(RotWord(w[i-1])) ^ Rcon[i/4]，Rcon[1..10]=01,02,04,08,10,20,40,80,1b,36。RotWord 是字节左旋一位：{w3[23:16], w3[15:8], w3[7:0], w3[31:24]}；**Rcon 异或到最高字节**：sub_word(rot) ^ {rcon, 24'b0}（写成 {24'b0, rcon} 会全错）。链式：nw0 = w0 ^ temp; nw1 = w1 ^

nw0; nw2 = w2 ^ nw1; nw3 = w3 ^ nw2;

- S-box 必须逐字节照抄下表，用 function automatic logic [7:0] sbox(input logic [7:0] a) + case 实现；**函数体内禁止使用 return**，用 sbox = 8'hxx; 给函数名赋值。xtime/rcon 同样用 function 实现（rcon 可在 function 里用 case 返回 8 位常量）。

```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00  63 7c 77 7b f2 6b 6f c5 30 01 67 2b fe d7 ab 76
10  ca 82 c9 7d fa 59 47 f0 ad d4 a2 af 9c a4 72 c0
20  b7 fd 93 26 36 3f f7 cc 34 a5 e5 f1 71 d8 31 15
30  04 c7 23 c3 18 96 05 9a 07 12 80 e2 eb 27 b2 75
40  09 83 2c 1a 1b 6e 5a a0 52 3b d6 b3 29 e3 2f 84
50  53 d1 00 ed 20 fc b1 5b 6a cb be 39 4a 4c 58 cf
60  d0 ef aa fb 43 4d 33 85 45 f9 02 7f 50 3c 9f a8
70  51 a3 40 8f 92 9d 38 f5 bc b6 da 21 10 ff f3 d2
80  cd 0c 13 ec 5f 97 44 17 c4 a7 7e 3d 64 5d 19 73
90  60 81 4f dc 22 2a 90 88 46 ee b8 14 de 5e 0b db
a0  e0 32 3a 0a 49 06 24 5c c2 d3 ac 62 91 95 e4 79
b0  e7 c8 37 6d 8d d5 4e a9 6c 56 f4 ea 65 7a ae 08
c0  ba 78 25 2e 1c a6 b4 c6 e8 dd 74 1f 4b bd 8b 8a
d0  70 3e b5 66 48 03 f6 0e 61 35 57 b9 86 c1 1d 9e
e0  e1 f8 98 11 69 d9 8e 94 9b 1e 87 e9 ce 55 28 df
f0  8c a1 89 0d bf e6 42 68 41 99 2d 0f b0 54 bb 16
```

五、目标微架构（面积优先，严格按此实现）

面积几乎全部来自 S-box：一个 S-box 约 1.5 k 面积单位，本设计**只允许 4 个。**为此 SubBytes 不并行做 16 个字节，而是每拍只做一列，并且**和密钥扩展的 SubWord 共用同一组 4 个 S-box。**

寄存器（不要多、也不要少）：s0..s3（4×32-bit 列寄存器）、w0..w3（4×32-bit 轮密钥，在线展开）、kt[31:0]（SubWord 暂存）、round[3:0]、phase[2:0]、busy、done、ready、ciphertext[127:0]。**不要** sb0..sb3 这类“SubBytes 结果寄存器”，SubBytes 结果直接写回 s。

S-box 的输入必须先用 phase 做多路选择，这样源码里只出现 4 个 sbox(...) 实例（in_a0..in_a3 是 8-bit 选择器，属于小逻辑，代价远低于 12 个额外 S-box）：

```
wire [7:0] in_a0 = (phase==3'd0) ? w3[23:16] :      // RotWord 的第 0 字节
                   (phase==3'd1) ? s0[31:24] :
                   (phase==3'd2) ? s1[31:24] :
                   (phase==3'd3) ? s2[31:24] : s3[31:24];
wire [7:0] in_a1 = (phase==3'd0) ? w3[15:8]  :
                   (phase==3'd1) ? s0[23:16] :
                   (phase==3'd2) ? s1[23:16] :
                   (phase==3'd3) ? s2[23:16] : s3[23:16];
wire [7:0] in_a2 = (phase==3'd0) ? w3[7:0]   :
                   (phase==3'd1) ? s0[15:8]  :
                   (phase==3'd2) ? s1[15:8]  :
                   (phase==3'd3) ? s2[15:8]  : s3[15:8];
wire [7:0] in_a3 = (phase==3'd0) ? w3[31:24] :      // RotWord 的第 3 字节
                   (phase==3'd1) ? s0[7:0]   :
                   (phase==3'd2) ? s1[7:0]   :
                   (phase==3'd3) ? s2[7:0]   : s3[7:0];
```

```
wire [7:0] sx0 = sbox(in_a0);
wire [7:0] sx1 = sbox(in_a1);
wire [7:0] sx2 = sbox(in_a2);
wire [7:0] sx3 = sbox(in_a3);
```

phase 每轮 0..5 循环，共 6 拍：

phase	动作
0	kt <= {sx0, sx1, sx2, sx3}（即 SubWord(RotWord(w3))，RotWord 已体现在上面的选择里）
1	w0<=nw0; w1<=nw1; w2<=nw2; w3<=nw3; 且 s0 <= {sx0, sx1, sx2, sx3}
2	s1 <= {sx0, sx1, sx2, sx3}
3	s2 <= {sx0, sx1, sx2, sx3}
4	s3 <= {sx0, sx1, sx2, sx3}
5	round<9：s0..s3 <= MixColumns(ShiftRows(s)) ^ w，round<=round+1，phase<=0；round==9：ciphertext <= ShiftRows(s) ^ w，done<=1、busy<=0、ready<=1、phase<=0

其中 nw 由 kt 组合产生：temp = kt ^ {rcon(round+1), 24'b0}，nw0=w0^temp; nw1=w1^nw0; nw2=w2^nw1; nw3=w3^nw2;。因为 phase=0 锁存的就是下一轮密钥，phase=1 再把它写进 w，所以 phase=5 的 AddRoundKey 用的是**新一轮**密钥。总延迟 10×6 = 60 拍（含握手约 61 拍），远低于 200 拍上限。

ShiftRows、MixColumns 全部是组合 wire：sr 由 s 的字节拼接（第四节四条式子），xtime 每列只算一次、在 4 个输出字节间共享，mc0..mc3 直接写成 wire [31:0] mc0 = {...} 的一体化表达式。不引入 128-bit 移位器、索引变量、循环或 always_comb。

自检硬指标：综合后 S-box 实例数必须为 4。如果写成 wire [31:0] sb0 = {sbox(s0[31:24]), sbox(s0[23:16]), sbox(s0[15:8]), sbox(s0[7:0])}; 并行处理 4 列，就会有 16 个实例、面积约 35 k，属于不合格（正确写法面积约 17 k）。

六、生成前自查

1. 只有一个模块，端口与第一节逐字一致，无重复声明。
2. function 内无 return；S-box 256 项逐项正确（抽查 00→63、01→7c、53→ed、ff→16）。
3. 3·b 写成 xtime(b) ^ b；ShiftRows 四个拼接式与第四节完全一致。
4. 第 10 轮无 MixColumns；Rcon 在最高字节；phase 每轮 0→5 循环、round 在 phase=5 且 round<9 时加一。
5. done 恰好一拍；ready 在 done 后下一拍为 1；复位同步低有效。
6. MixColumns 的 4 个输出只引用本列字节 b0..b3，没有混入相邻列的字节；out[3] 是 xtime(b0)^b0 ^ b1 ^ b2 ^ xtime(b3)，不是 b0^b1^b2^xtime(b3)^b3。
7. sbox(...) 只出现在 sx0..sx3 四行里（源码中恰好 4 处调用，综合 4 个实例），phase 通过 in_a0..in_a3 选择输入；最后一轮密文直接取 sr ^ w，第 1–9 轮 MixColumns 输入也直接取 sr，都没有再调 sbox。
8. 口算校验：key=000102...0f、pt=00112233...eeff → ciphertext=69c4e0d86a7b0430d8cdb78070b4c55a；第 1 轮

AddRoundKey 后应为 89d810e8855ace682d1843d8cb128fe4。

只输出最终 RTL 源码。

Prompt 设计基本思想

1. 接口与时序先写死：Prompt 第一段逐字给出端口声明，并明确「同步低有效复位、done 只高一拍、done 之后下一拍 ready=1 可背靠背接收、约 60 拍出结果（上限 200 拍）」。这四句直接对应教师 TB 的 5 组检查（复位、单拍 done、ready 恢复、背靠背、中途复位），避免模型自行发挥成异步复位或完成信号拉高多拍。
2. 字节序写成位段：FIPS 大端下第 0 列占最高 32 位，因此 s0 取 plaintext[127:96] 而不是 plaintext[31:0]。实测模型把 plaintext[31:0] 当第 0 列时，握手时序全部正确、5 个向量密文全错（hw1-r2）；只改这一条赋值即 PASS。Prompt 里把这段标为「最容易错的一步，务必逐字照抄」并写明错误现象。
3. 面积几乎全由 S-box 决定：一个 S-box 约 1.5 k 面积单位。把「4 个数据 S-box + 4 个密钥 S-box」压到「4 个 S-box 数据/密钥共用」，面积从 35.0 k 降到 17.1 k（约一半）。做法是先用 phase 对 4 个 8-bit 输入做多路选择（in_a0..in_a3），S-box 输出 sx0..sx3 再按 phase 分别写进 kt 或某一列的 s——选择器是几十门的小逻辑，比多 4 个 S-box 便宜得多。
4. 微架构是显式的 6 拍状态表：phase=0 算 SubWord 存 kt；phase=1 锁存下一轮密钥 w 并做第 0 列 SubBytes；phase=2..4 做其余三列；phase=5 做 ShiftRows+MixColumns+AddRoundKey（最后一轮只做 SR+ARK 并输出）。10 轮 ×6 拍 = 61 拍，仍在 200 拍上限内。把表直接写给模型，比只描述「按列串行」可靠得多。
5. 组合逻辑全部 wire+assign：rotw、temp、nw0..nw3、sr0..sr3、xtime、mc0..mc3 都写成一体化表达式，禁止 always_comb 复用中间变量（否则 iverilog 报 Variable 'a0' cannot have multiple drivers，hw1-r11 即栽在此处），也不引入 128-bit 移位器与索引变量。
6. 三个高频错误写成显式反例：①3·b 必须是 xtime(b)^b（写成 b^b 恒为 0）；②MixColumns 的 out3 是 (xtime(b0)^b0)^b1^b2^xtime(b3)，写成 b0^b1^b2^xtime(b3)^b3 会让每列最后一个字节全错（hw1-v6r1 即因此失败，且握手一切正常、极难从报告看出）；③最后一轮直接取 sr^w，不得对 sr 再调 sbox（否则多 16 个实例、面积涨到 52 k，hw1-r17/r26）。MixColumns 还给了可直接照抄的四行 wire 模板。
7. 输出形式约束：只允许输出一个 module 的完整源码，禁止 include / $readmemh / DPI / IP 例化 / testbench / 第二个模块。既满足「单模块」评分要求，也让 ai_eda 的抽取器可直接落盘编译（completion 约 6.2 k tokens）。
8. 与课程参考实现对照：同一套 iverilog + yosys(gscl45nm) + OpenSTA 流程下，课程手写实现（20 个 S-box、全并行、10 拍）area=35803.37、total_power=0.0426 W；本方案 area=17091.44、total_power=0.0099 W，面积小 52%、功耗低 77%，代价是延迟从 10 拍增加到 61 拍（上限 200 拍，仍有 3 倍余量）。
9. 迭代与收敛：v1 全错（35419）→ v2 加 5 拍微架构仍全错（34955）→ v3 修正列映射 + wire/assign，15 次 12 次 PASS（80%）→ v4 补轮函数边界，15 次全过（35.0 k）→ v5 尝试 4 S-box 但模型把 SubBytes 展开成 16 个实例（36.8 k）→ v8 给出显式 phase 表、S-box 输入多路选择和 MixColumns 四行模板后，8 次运行全部 PASS，面积稳定在 17.1 k。失败样本全部保留在 runs/ 下。

评分规则简述

评分维度	判定要求
功能正确性	以教师 AES TB 的通过结果为前提；未通过时不进入 PPA 比较。
Prompt 质量	Prompt 应完整、可复现，并清楚约束接口、算法要点、时序和输出形式。
工程与指标	提交可追溯的运行记录，比较 tokens、耗时、面积和功耗，并解释最终方案选择。

运行与 PPA 记录

运行编号	模型与主要参数	教师 TB	消耗 tokens	消耗时间	面积	总功耗	备注
1	DeepSeek-V4.1-Flash-1017 T=0.2 / max_tokens=12000 hw1-v8r4	PASS 5 组向量	11135 (4895+6240)	23.3 s +6 s TB/PPA	17140.71	9.83 mW	61 拍出结果；复位/单拍 done 全过
2	DeepSeek-V4.1-Flash-1017 T=0.2 / max_tokens=12000 hw1-v8r2	PASS 5 组向量	11127 (4895+6232)	21.7 s +6 s TB/PPA	17091.44	9.88 mW	面积最小 17091.44，最终采用
3	DeepSeek-V4.1-Flash-1017 T=0.2 / max_tokens=12000 hw1-v8r8	PASS 5 组向量	11053 (4895+6158)	31.6 s +6 s TB/PPA	17119.13	10.30 mW	功耗 10.33 mW；背靠背正常
最终采用	同 run 2 hw1-v8r2	PASS 5 组向量	11127 (4895+6232)	21.7 s	17091.44	9.88 mW	internal 6.35 switching 3.45 leakage 0.080 mW

重复性验证：最终版 Prompt（v8）定型后，用同一模型与参数独立运行 8 次（run_id hw1-v8r1 ~ hw1-v8r8），8 次通过教师 TB，成功率 100%，面积 17091.44 ~ 17190.93（波动 0.6%）、总功耗 9.8 ~ 10.3 mW。中间版本 v5（4 S-box 但 SubBytes 被展开，36.8 k）、v6（MixColumns out3 写错）、v7（phase=5 又异或了 nw，密钥多推一轮）各有 1~4 次失败，样本同样保留在 runs/ 下。

需要说明的是：面积从 35.0 k 降到 17.1 k 靠的是「4 个 S-box 由数据通路和密钥扩展共用」，而这一结构对模型并不友好——v5 把 SubBytes 展开成 16 个并行实例（36.8 k），v6 把 MixColumns 的 out3 写错，v7 则在 phase=5 又异或了一次 nw（此时 w 已是新一轮密钥，等于密钥多推一轮），三版共 9 次运行全部失败。直到 Prompt 里给出 phase 状态表、S-box 输入多路选择器、MixColumns 四行模板，并明确「phase=5 只用 w」，v8 才稳定到 8/8。

填写说明：tokens 建议记录 API usage 的总量；时间记录从发起生成到得到该次 TB 结果的墙钟时间；面积与功耗仅在教师 TB 通过且使用固定课程工具链时填写。

验证结果入口

复现命令（在项目根目录执行，需先 pip install -r requirements.txt）：
.venv/bin/python -m ai_eda run --task homework_aes128 --prompt homework/H1_custom.md --run-id hw1-v8r2 --max-tokens 12000

运行记录（Prompt、原始回答、生成 RTL、教师 TB 报告、VCD、PPA 指标）：
runs/prompt/homework_aes128/hw1-v8r2/、hw1-v8r4/、hw1-v8r8/（v8 共 8 次：hw1-v8r1 ~ hw1-v8r8）
每个目录包含 prompt.md、raw_response.txt、generated.sv、test_report.json、activity.vcd、metadata.json。

教师 TB 结果：generated.sv 在 content/tasks/homework_aes128/tb.sv 下输出 LAB_RESULT: PASS (5 vectors)；
功耗 TB 为 content/tasks/homework_aes128/power_tb.sv（12 组向量）。

PPA 流程：iverilog -g2012 仿真取 activity.vcd → yosys 以 gscl45nm.lib 综合得到面积 → OpenSTA 读 VCD 估算功耗；
数值见 test_report.json 的 metrics 字段（library=gscl45nm.lib，top_module=aes128，power_method=OpenSTA gate-level VCD estimate）。

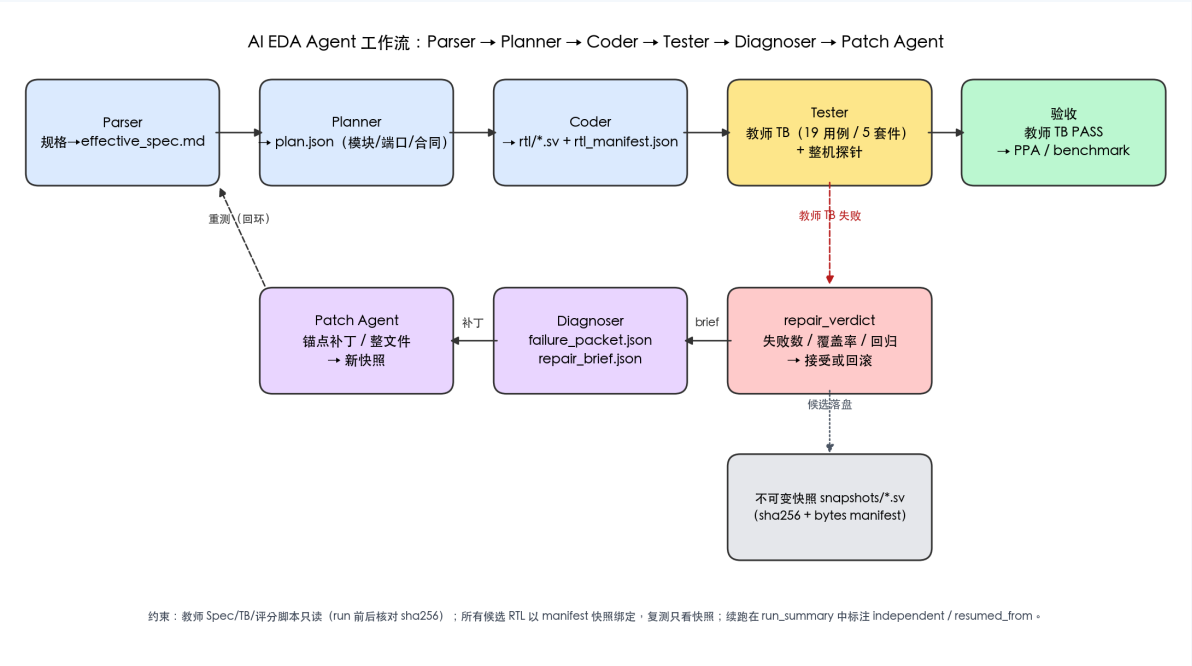
Prompt 源文件：content/prompts/homework/H1_custom.md；本报告的表格由 work/fill_report_hw1.py 从上述记录自动填入。

作业二 EDA Agent 工作流重构

任务概述：选择一个 RISC V 难度等级，改造 Agent 的规划、诊断、补丁和交接流程。重点说明如何减少 Modifier 的无关上下文，如何拆分 Diagnoser 和 Patch Agent，以及如何通过教师 TB、PPA 与 benchmark 证明改进有效。

选择的难度等级	请勾选或填写	目标项目	教师 TB 最终状态
L0 基础五级流水线	☐	—	未作为目标
L1 I D Cache	☑	rv32i5_cache	PASS：空白独立生成 13/54 = 24%，单次最快 5 次调用 / 121 s / ¥0.35
L2 Cache 加分支预测	☑	rv32i5_predict	PASS：agent-20261112T131259447607-cbcc60（续跑，34→0，9 次调用 / 215 s）
L3 Cache 预测加乱序	☑ (主目标)	rv32i5_ooo	PASS：agent-20261112T060026252331（续跑，33→0，7 次调用 / 145 s）

设计的 Agent 流程图



（上图为本实现的节点与失败回环；命名对照与各 artifact 见下方说明框。）

流程图说明

主链 Parser → Planner → Coder → Tester 对应原项目同名节点；失败回环里新增 repair_verdict（判决器，决定接受或回滚），并把原先内联在 Tester/Modifier 的失败分析拆成两个角色：Diagnoser 只出证据与约束、不改 RTL，Patch Agent 只读 brief 与允许文件、出补丁。命名对照：原「Modifier」= 本实现的 Patch Agent。判据只有教师 TB（19 个用例 / 5 个套件），探针只做定位。每个候选 RTL 都落成不可变快照（manifest 含每文件 sha256 + 字节数），复测只针对快照。

子 Agent 功能与交接产物

Agent	主要功能	接收的文本或文件	交给下游的文本或文件	参数或 Prompt 要点
Parser	把教师 spec/讲义解析成可验收条款	教师 spec（只读）	effective_spec.md（含章节号与验收口径）	只抽取、不改写；禁止自创接口
Planner	冻结模块划分、顶层端口、级间边界与流水线控制合同	effective_spec.md + 难度 tier 合同	plan.json（模块表/端口/冻结组件/pipeline_control_contract）	端口由合同生成；重复端口自动去重；if_id_ready 方向错误会带「实际 vs 期望」报错重试
Coder	按 plan 生成/重写 RTL 模块	plan.json + 架构蓝图 + 踩坑清单	rtl/*.sv + rtl_manifest.json	模块头必须与计划一致；先声明后使用；所有状态寄存器在声明处给初值（教师 TB 在 time 0 就查 dmem_valid）
Tester	编译 + 教师 TB + PPA + 定位探针	rtl_manifest（快照绑定）	test_report.json + metrics（面积/功耗）+ performance（benchmark）+ 探针观测	唯一判据是教师 TB；探针在失败时按需挂载（L1 5 个、L3 15 个）
Diagnoser	把失败报告压缩成 failure packet 与 repair brief	test_report.json + 票据 + 教师用例原文	failure_packet.json、repair_brief.json（hypothesis/constraints/verification_goal/teacher_test_source）	只读证据、不动 RTL；机械诊断被证伪（上一轮零效果）时强制改用模型诊断
Patch Agent	按 brief 出补丁并自检	repair_brief + 允许修改的文件 + 接口合同	changed files → 新快照（snapshots/*.sv + manifest）	路径白名单；锚点补丁失败可退化为整文件；编译期机械修复（重复声明/端口/双驱动）最多三轮；预算内重试
其他	判决器 + 不可变快照	候选 report 与基线 report；RTL 文件	repair_evaluation.json（接受/回滚 + 理由）、manifest（每文件 sha256+bytes）	失败测试数优先；断言数翻倍判负；续跑在 run_summary 标注 independent / resumed_from

失败报告的压缩：Tester 输出的是教师 TB 的原始逐条 check（一次失败可上万条），Diagnoser 先按「测试 × 失败签名」聚合，只保留失败的测试、每测试的前若干条证据、以及该测试对应的教师用例源码片段（teacher_test_source，≤6000 字符），形成 failure_packet.json；再按当前票据（哪一类失败最值得修）

挑出小表，形成 repair_brief.json。brief 里显式带三类内容：①证据（失败计数、首次失败行、探针观测）；②不可破坏约束（模块端口冻结、等待期间请求必须保持、ECALL 不得 flush 等，按票据类型给出）；③验证目标（本次要复现的用例、必须不回退的用例集合、接受标准）。Patch Agent 因此不需要读整个工程：它只拿到 brief、允许修改文件的完整源码与接口合同，输出补丁后由 Tester 在快照上复测；两轮无进展时 brief 会追加「零效果警报」，要求换机制或整份重写。所有交接产物都是文件引用 + 内容哈希：handoff 里记录 run_id、轮次、文件引用、字节数与 sha256，复测绑定的快照 manifest 记录每个 RTL 文件的 sha256，判决器只比较同一快照体系下的报告。

Agent 运行与 PPA 记录

运行编号	项目等级	教师 TB	面积	总功耗	消耗 tokens	消耗时间	修复轮次与备注
1	L1	PASS	144485.27	0.0256 W	80,357（56,948+23,409）	121 s	rv32i5_cache；2 轮 tester，5 次调用，空白独立生成
2	L1	PASS	138053.51	0.0224 W	226,915（182,719+44,196）	301 s	rv32i5_cache；9 轮 tester，18 次调用，空白独立生成
3	L2	PASS	163371.78	0.0363 W	127,090（109,872+17,218）	215 s	rv32i5_predict；续跑自 agent-20261112T125711019179（34→0），9 次调用
最终采用	L3	PASS	291482.70	0.0336 W	≈140,000*	145 s	rv32i5_ooo；续跑自 agent-20261112T055150850447，33→20→20→19→5→0，7 次调用 / 6 轮 tester

Benchmark 性能评估

Benchmark	通过状态	周期数	CPI	分支误预测	乱序旁路事件	其他指标与说明
trained_branch64	PASS	407（132 条指令）	3.083	2 / 64	—	IPC 0.324
teaching_crc32_64bytes	PASS	9,946（3,185 条指令）	3.123	302 / 1,088	4	IPC 0.320
teaching_array_sum64	PASS	1,023（324 条指令）	3.157	2 / 64	30	IPC 0.317
teaching_branch_predict64	PASS	963（293 条指令）	3.287	35 / 128	—	IPC 0.304
teaching_ooo_overlap64	PASS	2,187（516 条指令）	4.238	2 / 64	75	IPC 0.236

说明：上表取自 L3 通过运行 agent-20261112T060026252331 的 run_summary.json → performance.benchmarks（教师 PPA/benchmark 输出原文），其中「乱序旁路事件」= ooo_bypass 计数，CPI/IPC 由教师工具链按 cycles/instructions 给出；L1/L2 运行不产生 ooo 指标，按 N/A 处理。

性能结论与证据位置

主要性能瓶颈与重构效果：功能上 L3 一版通过教师 5 个套件（base 254,147 / cache 1,898 / predictor 1,227 / ooo 1,137 / performance_correctness 42,379 条全 PASS），L2 版 260,531 / 2,054 / 1,233 / 0 / 42,874 条全 PASS；面积/功耗 L3 291,482.70、0.0336 W，L2 163,371.78、0.0363 W。瓶颈：(a) I-Cache 重填与 flush 的握手；(b) 存储字节车道（SB/SH 未按 addr[1:0] 生成 be、未复制 dmem_wdata）；(c) L3 停机/冲刷边界。重构效果是可诊断性：failure packet + brief（证据/约束/验证目标）→ Patch Agent 只读 brief 与允许文件即可出补丁。原始日志：evidence/<run_id>/run_summary.json 的 performance.benchmarks 与 selected_test_report.json。

* L3 那次运行的流式 usage 未回传（早期客户端未开 include_usage），按同口径链上三次运行估算：agent-20261112T050852828836（500,616+85,679）、…055150850447（132,696+9,605）、…060026252331（94,957+7,980），合计 prompt 728,269 / completion 103,264 tokens，约 ¥2.28。

附：L3 的可追溯经历与复现入口

L3 并不是一次生成就通过的。仓库 evidence/ 与 runs/ 里保留了这条链的每一步（均为 agent 自己跑出来的运行，未手工改 RTL）：

① agent-20261112T050852828836：空白独立生成（independent=true），6 轮 tester 后停在 49 条失败，被 icache 独立探针拦下（该探针判 icache 端口/重填行为不合格）；② agent-20261112T054209658666：从上一轮快照续跑（13 次调用），49 → 33；③ agent-20261112T055150850447：继续续跑（9 次调用），33 → 3；④ agent-20261112T060026252331：继续续跑（7 次调用、6 轮 tester），33 → 20 → 20 → 19 → 5 → 0，教师 TB 五个套件全部 PASS，面积 291,482.70、功耗 0.0336 W。整条链合计 53 次模型调用、约 ¥2.28。

另外记录两次失败尝试，用于说明边界：11-12 晚用同一工作流从 3 条那份快照复跑，得到 3 → 20 → 19 → 7 → 6 → 6 → 6 与 6 → 1 → 1，最后剩下的 1 条是 simulation_timeout（把教师仿真墙钟上限从 90 s 提到 200 s / 240 s 仍未结束），说明那一版 RTL 在 L3 套件后段真的停住，不是断言算错；此时报告按未通过处理，不把它写成通过。

复现入口：仓库根目录 `python scripts/run_cpu_agent.py --level 3 --resume-from <run_id> --max-calls 25 --max-repairs 8`，环境变量 `EDA_MODEL=DeepSeek-V4.1-Flash-1017`、`EDA_BASE_URL=https://llmapi.paratera.com/v1`、`EDA_SIM_TIMEOUT`（教师仿真墙钟，默认按项目配置）。独立空白运行去掉 `--resume-from` 即可；所有运行的汇总可用 `python scripts/report_runs.py --all` 输出（含 tokens 与花费）。